

Trabalho Prático em React - “Batalha Naval Avançada”

> Âmbito

Este trabalho prático (TP) tem como objetivo o desenvolvimento de uma aplicação em **React JS**, permitindo aos alunos aplicar os conhecimentos adquiridos ao longo da unidade curricular. Como tal, devem demonstrar o domínio em **React** e nas tecnologias necessárias ao desenvolvimento da aplicação, nomeadamente o JavaScript moderno, HTML e CSS. Todos os detalhes encontram-se descritos nas secções seguintes, recomendando-se a leitura integral do presente documento.

O TP é obrigatório e tem um peso de **8 valores** da nota final da unidade curricular. Valoriza-se a autonomia, organização, qualidade técnica e criatividade da solução apresentada.

A submissão de trabalhos de terceiros, total ou parcialmente, ou trabalhos gerados integralmente por IA sem compreensão do código, implica 0 na nota final e ativação dos mecanismos legais associados a plágio académico.

Considerações Gerais:

- Os alunos devem organizar-se **em grupos de 2 ou 3 alunos, e 1 aluno, desde que devidamente justificado e previamente autorizado.**
- Todos os elementos devem participar ativamente no desenvolvimento do TP.
- O TP deverá seguir as orientações de implementação ([ver condições de implementação](#)).
- O TP deverá ser submetido até às **8h de 1 de junho de 2026** ([ver condições de entrega](#)).
- O trabalho terá uma defesa obrigatória, a realizar **entre os dias 1 e 5 de junho**, mediante pré-inscrição no InforEstudante. **A não comparência** à defesa na data (dia e hora) combinada, implica a anulação do trabalho e consequente reprovação do aluno à disciplina. Na defesa, qualquer elemento poderá ser questionado sobre qualquer parte do trabalho, bem como solicitado alterações ao código apresentado.
- Os alunos que pretendam manter a nota do TP obtida em 2024/2025, devem submeter os dados no formulário, secção “**Submissão de Trabalhos**”, cujo título “**Manter nota do TP de LS**”. Apenas é possível manter notas superiores ou iguais a **50%**.
- O não cumprimento das regras detalhadas no enunciado, implica penalização na nota do TP.

> **Tema: Batalha Naval Avançada**

O tema do Trabalho Prático é uma **variante** do jogo clássico “**Batalha Naval**”, com uma vertente tática e de gestão de recursos críticos. O jogo decorre entre dois adversários (o Jogador e o Computador), onde cada um possui uma frota de navios distribuída num tabuleiro. O objetivo é destruir toda a frota adversária, gerindo os recursos disponíveis.

A frota será distribuída num tabuleiro quadrado de dimensão mínima 10x10. Após o posicionamento dos navios, o jogo inicia, com os jogadores a atacarem alternadamente uma célula do adversário até que toda a frota de um deles seja destruída.

Nesta versão de **Batalha Naval**, existem características especiais:

- **Gestão de Combustível** - O jogador inicia com **100 unidades de combustível** (valor máximo permitido). Cada disparo consome **5 unidades**. Se acertar num navio inimigo, recupera 10 unidades (até ao limite de 100). Se o combustível chegar a 0, o jogador perde imediatamente o jogo.
- **Cronómetro de turno** - O jogador tem um limite de 15 segundos para realizar cada jogada. O cronómetro deve ser visível, atualizar em tempo real em contagem decrescente (15 a 0). Se o tempo esgotar, o turno passa automaticamente para o computador e o jogador é penalizado em 5 unidades de combustível.
- **Radar Inteligente (recompensa)** - A opção de "Radar" fica disponível apenas sob condições de mérito: o jogador deve acertar num navio inimigo numa jogada realizada em menos de 3 segundos. Ao ativar o radar (ação livre que não consome o turno nem combustível), o sistema destaca no tabuleiro do computador uma área de 2x2 que contenha, pelo menos, uma parte de um navio ainda não atingida. O radar é de uso único por cada ativação obtida.

> **Funcionalidades Obrigatórias**

O jogo deverá ser implementado em **React 19**, utilizando as melhores práticas (Componentes Funcionais, Hooks) e disponibilizar, pelo menos, as seguintes funcionalidades:

- **Configuração Inicial**: Solicitar o nome do jogador antes do início da partida.
- **Posicionamento da Frota**: Apresentar o tabuleiro vazio e permitir o posicionamento manual da frota do jogador por interação de clique. Os navios podem ser posicionados horizontal ou verticalmente, não sendo permitida a sobreposição nem o posicionamento fora dos limites do tabuleiro. A frota obrigatória é composta por:

- 1 navio de tamanho 5
- 1 navio de tamanho 4
- 2 navios de tamanho 3
- 2 navios de tamanho 2

- **Configuração do Computador:** Deverá existir uma lista/menu que permita ao utilizador seleccionar entre 3 tabuleiros com frota pré-definida para o computador, ou escolher uma opção de "frota aleatória" (onde o sistema gera um posicionamento válido de forma autónoma).
- **Modo Debug/Testes:** Deverá existir uma caixa de seleção (checkbox) para apresentar/esconder a frota invisível do computador. Isto servirá para facilitar a correção, testes rápidos e verificação do funcionamento correto do jogo por parte do docente.
- **Interface de Jogo:** O sistema deverá distinguir e apresentar visualmente as seguintes fases das células ao longo do jogo:
 - Água (Não explorada)
 - Tiro Falhado (Água explorada)
 - Acerto (Parte do navio atingida)
 - Navio Destruído (Totalmente afundado)
- **Painel de Informações:**
 - Apresentar o **Cronómetro de Turno** (15s decrescentes).
 - Apresentar o **Nível de Combustível** atual, atualizando-o de acordo com os disparos e penalizações.
 - Mostrar o botão do **Radar** como "Disponível" ou "Indisponível", dependendo da satisfação da regra de mérito.
- **Inteligência do Computador:** Quando é o computador a atacar, a implementação da sua lógica de ataque terá dois níveis de valorização:
 - **Nível Básico (Menos valorizado):** O computador ataca sempre numa posição puramente aleatória (desde que não tenha sido previamente atacada).
 - **Nível Avançado (Mais valorizado):** O computador ataca aleatoriamente até encontrar um navio. Assim que acerta numa célula com navio, os ataques seguintes deverão ser feitos nas posições adjacentes (horizontal e vertical) até que esse navio seja totalmente destruído. Só depois de o afundar é que retoma a procura aleatória.
- **Fim de Jogo:** Quando houver um vencedor (ou o combustível acabar), deverá ser apresentado um ecrã final com:
 - O Vencedor.
 - O número de jogadas efetuadas.
 - Um botão para "Jogar Novamente", que reinicia os estados todos para uma nova partida.

> **Implementação**

O trabalho deverá ser implementado em **React 19**, recorrendo exclusivamente a *Componentes funcionais e Hooks React*. É dada a liberdade ao aluno para seguir a estratégia que considerar melhor, no entanto, especificam-se abaixo as **características e limitações** obrigatórias:

- **Base de Projeto:** Os alunos devem usar **obrigatoriamente** base do projeto fornecido nas aulas práticas;
- **Arquitetura de Componentes:** A aplicação não pode ser um ficheiro monolítico. É obrigatória a separação lógica da interface em múltiplos componentes com responsabilidades bem definidas. Exige-se, no mínimo, a divisão da interface nos seguintes componentes (ou equivalentes):
 - Componente para o Tabuleiro (**Board**)
 - Componente para a Célula individual do tabuleiro (**Cell / Square**)
 - Componente para o Painel de Informações (**Dashboard / ControlPanel** - para gerir cronómetro, combustível e radar)
 - Componente para a fase de Posicionamento da Frota (**Setup**)
 - Componente para o Ecrã Final (**GameOver / Results**)
- A **gestão de estados** (useState) não deve ser centralizada inteiramente no componente principal (App).
- **Não é permitido** o uso de useReducer, Context API ou bibliotecas externas (Redux, etc).
- **Não é permitida** manipulação direta do DOM, sob pena de anulação da cotação do respetivo requisito. São expressamente proibidos:
 - getElementById (exceto nó raiz da aplicação)
 - querySelector
 - addEventListener manual (devem usar os eventos sintéticos do React, como onClick)
- **Não são permitidas** bibliotecas de componentes React pré-feitos (caso contrário será considerado que o aluno não implementou esse componente nem as respetivas funcionalidades). São proibidos:
 - Material UI
 - Ant Design
 - React Bootstrap / Reactstrap
- Fica à responsabilidade do aluno a especificação da interface da aplicação. **São permitidas bibliotecas** utilitárias de CSS para auxiliar no layout, tais como:
 - Bootstrap
 - Tailwind CSS

- Todos os componentes necessários à implementação do jogo devem ser criados pelos alunos. Se assim o entenderem, **podem reaproveitar e adaptar componentes criados previamente nas aulas práticas.**

> **Relatório**

O TP deve ser acompanhado de um pequeno relatório técnico, com o máximo de 3 páginas, onde devem **apresentar obrigatoriamente o diagrama de componentes**, as funcionalidades dos componentes implementados e as soluções utilizadas no desenvolvimento da aplicação.

> **Condições de Entrega**

- O TP deverá ser submetido **pelo *inforestudiante***, até às **8h** do dia **1 de junho de 2026**.
- A entrega **deve ser efetuada apenas por um elemento do grupo**, devendo indicar no momento da submissão os outros elementos do grupo de trabalho.
- **Submissão em ficheiro ZIP** com nome **LS2526_num1_num2_num3.ZIP**, contendo :
 - Projeto React completo (sem pasta *node_modules* nem ficheiro *package-lock.json*)
 - Relatório PDF
- **Trabalhos práticos submetidos depois do horário terão uma penalização de 5% da nota final, por cada hora de atraso. Não se aceitam trabalhos práticos enviados por email.**

> **CrITÉrios de Avaliação**

- **Aspeto Gráfico (5%):** Qualidade visual, disposição dos elementos (Informações, tabuleiros lado a lado) e clareza da interface.
- **Setup Inicial (7.5%):** Solicitar e apresentar o nome do jogador, destacando visualmente de quem é a vez em cada jogada.
- **O Tabuleiro e Estados (10%):** Apresentar os tabuleiros e gerir corretamente a renderização visual dos 4 estados das células à medida que o jogo avança (Água/Nevoeiro, Tiro Falhado, Acerto, Navio Destruído).
- **Posicionamento da Frota do Jogador (15%)** O jogador seleciona a orientação (horizontal/vertical) num menu/botão e clica no tabuleiro para colocar o navio. O sistema valida limites e sobreposições, rejeitando posições inválidas.
- **Configuração do Computador e Debug (10%)** - Permitir selecionar entre as 3 frotas pré-definidas ou gerar frota aleatória para o Computador. Incluir a Checkbox funcional que mostra/esconde a frota invisível do PC para efeitos de teste.

- **Gestão de Tempo de Jogada (10%)** – Apresentar o cronómetro de 15 segundos em contagem decrescente para cada jogada do jogador. Se chegar a 0, o turno tem de passar automaticamente e aplicar a penalização.
- **6. Gestão de Combustível (10%)** – Início a 100 unidades. Descontar corretamente 5 unidades por disparo e 5 por penalização de tempo. Somar 10 unidades por acerto (respeitando o teto máximo de 100) e ditar a derrota imediata se chegar a 0.
- **Radar Inteligente (10%)** – Verificar corretamente se a jogada demorou menos de 3 segundos e resultou em acerto. Desbloquear o botão e, ao clicar, destacar de forma precisa uma grelha 2x2 válida no tabuleiro inimigo.
- **Inteligência do Computador (Máx: 17.5%)**
 - **Opção 1 (5% - Nível Básico):** O computador efetua sempre os seus disparos numa posição puramente aleatória (validando apenas para não atirar onde já atirou).
 - **Opção 2 (17.5% - Nível Avançado):** O computador ataca aleatoriamente, mas mal acerta num navio, guarda esse estado e passa a disparar nas posições adjacentes até afundar o navio na totalidade, regressando depois ao modo aleatório.
- **Conclusão do Jogo (5%)**
 - Identificação correta do fim de jogo (seja por destruição total da frota ou por falta de combustível).
 - Apresentação do ecrã final com o Vencedor, número de jogadas totais e botão funcional para "Jogar Novamente".
- ***Critérios Gerais e Fator de Modulação (A aplicar à nota final)***
 - **Organização e Arquitetura do Código:** Estruturação da aplicação em componentes React lógicos e reutilizáveis. Uso adequado das *props* e ausência de duplicação de código.
 - **Defesa do Aluno:** Capacidade de explicar a lógica implementada, justificar as escolhas arquiteturais e alterar pequenos blocos de código se solicitado no momento da avaliação.